Productivity Tracker API Documentation

Overview

The Productivity Tracker REST API is a simple REST-style web service for managing personal productivity data. It provides endpoints for creating, reading, updating, and deleting **tasks** and **habits**. All communication uses JSON over HTTP, and the API is implemented using plain Django views and models (without Django REST Framework).

Base URL

All API endpoints are exposed under the `/api/` path.

For local development:

```
http://127.0.0.1:8000/api/
```

Data Models

Task

The **Task** resource represents a single to-do item with a title, optional description, optional due date, priority, and status.

Field	Type	Description
`id`	integer	Unique identifier for the task (assigned by the server).
`title`	string	Short title for the task.
`description`	string	Optional longer description of the task (may be an empty string).
`due_date`	date or null	Optional due date in `YYYY-MM-DD` format; `null` if not set.
`priority`	string	Priority level: one of `low`, `medium`, `high`.
`status`	string	Current status: one of `not_started`, `in_progress`, `completed`.
`created_at`	datetime	Timestamp when the task was created (ISO 8601 string).

Habit

The **Habit** resource represents a recurring activity the user wants to track over time.

Field	Type	Description
`id`	integer	Unique identifier for the habit (assigned by the server).
`name`	string	Name of the habit.

Field	Type	Description
`frequency`	string	How often the habit occurs: one of `daily`, `weekly`.
`streak`	integer	Non-negative integer representing the current streak count.
`completed_today`	boolean	Whether the habit has been completed on the current day.
`created_at`	datetime	Timestamp when the habit was created (ISO 8601 string).

Task Endpoints

GET `/api/tasks/`

Retrieve a list of all tasks. Results are returned as a JSON array of task objects, ordered from most recently created to oldest.

Parameters

- Optional query parameters for filtering are documented in the **Filtering** section.

Example Response

Status: `200 OK`

```
[
  {
    "id": 1,
    "title": "Complete coursework",
    "description": "Finish the API documentation for the project",
    "due_date": "2026-03-15",
    "priority": "high",
    "status": "in_progress",
    "created_at": "2026-03-10T14:30:00Z"
  },

  {
    "id": 2,
    "title": "Review lecture notes",
    "description": "",
    "due_date": null,
    "priority": "medium",
    "status": "not_started",
    "created_at": "2026-03-09T09:00:00Z"
  }
]
```

GET `/api/tasks/{id}/`

Retrieve a single task by its `id`.

Path Parameters

- `id` (integer): Identifier of the task to retrieve.

Example Response

Status: `200 OK`

```
{
  "id": 1,
  "title": "Complete coursework",
  "description": "Finish the API documentation for the project",
  "due_date": "2026-03-15",
  "priority": "high",
  "status": "in_progress",
  "created_at": "2026-03-10T14:30:00Z"
}
```

If the task does not exist, the API returns:

- Status: `404 Not Found`

POST `/api/tasks/`

Create a new task. The request body must be valid JSON.

Request Body

```
{
  "title": "Write report",
  "description": "Finish coursework report",
  "due_date": "2026-03-15",
  "priority": "high",
  "status": "not_started"
}
```

- `title` is required.
- `description` is optional (defaults to an empty string).
- `due_date` is optional and must be a valid date string (`YYYY-MM-DD`) if provided.
- `priority` is optional and must be one of `low`, `medium`, `high`. If omitted, it defaults to `medium`.
- `status` is optional and must be one of `not_started`, `in_progress`, `completed`. If omitted, it defaults to `not_started`.

Example Successful Response

Status: `201 Created`

```
{
  "id": 3,
  "title": "Write report",
  "description": "Finish coursework report",
  "due_date": "2026-03-15",
  "priority": "high",
  "status": "not_started",
  "created_at": "2026-03-10T15:00:00Z"
}
```

If validation fails, the API returns `400 Bad Request` with an error body as described in **Error Responses**.

PUT `/api/tasks/{id}/`

Update an existing task. Only the fields provided in the request body are updated.

Path Parameters

- `id` (integer): Identifier of the task to update.

Example Request Body

```
{
  "status": "completed",
  "priority": "high"
}
```

Example Successful Response

Status: `200 OK`

```
{
  "id": 1,
  "title": "Complete coursework",
  "description": "Finish the API documentation for the project",
  "due_date": "2026-03-15",
  "priority": "high",
  "status": "completed",
  "created_at": "2026-03-10T14:30:00Z"
}
```

If the task does not exist, the API returns `404 Not Found`. If the payload is invalid, it returns `400 Bad Request`.

DELETE `/api/tasks/{id}/`

Delete a task by its `id`.

Path Parameters

- `id` (integer): Identifier of the task to delete.

Example Successful Response

Status: `204 No Content`

Response body: *(empty)*

If the task does not exist, the API returns `404 Not Found`.

Habit Endpoints

GET `/api/habits/`

Retrieve a list of all habits. Results are returned as a JSON array of habit objects, ordered from most recently created to oldest.

Parameters

- Optional query parameters for filtering are documented in the **Filtering** section.

Example Response

Status: `200 OK`

```
[
  {
    "id": 1,
    "name": "Drink water",
    "frequency": "daily",
    "streak": 5,
    "completed_today": true,
    "created_at": "2026-03-05T08:00:00Z"
  },

  {
    "id": 2,
    "name": "Exercise",
    "frequency": "weekly",
    "streak": 2,
    "completed_today": false,
    "created_at": "2026-03-01T10:15:00Z"
  }
]
```

GET `/api/habits/{id}/`

Retrieve a single habit by its `id`.

Path Parameters

- `id` (integer): Identifier of the habit to retrieve.

Example Response

Status: `200 OK`

```
{
  "id": 1,
  "name": "Drink water",
  "frequency": "daily",
  "streak": 5,
  "completed_today": true,
  "created_at": "2026-03-05T08:00:00Z"
}
```

If the habit does not exist, the API returns `404 Not Found`.

POST `/api/habits/`

Create a new habit. The request body must be valid JSON.

Request Body

```
{
  "name": "Drink water",
  "frequency": "daily",
  "streak": 0,
  "completed_today": false
}
```

- `name` is required.
- `frequency` is optional and must be one of `daily`, `weekly`. If omitted, it defaults to `daily`.
- `streak` is optional and must be a non-negative integer. If omitted, it defaults to `0`.
- `completed_today` is optional and must be a boolean. If omitted, it defaults to `false`.

Example Successful Response

Status: `201 Created`

```
{
  "id": 3,
  "name": "Drink water",
  "frequency": "daily",
  "streak": 0,
  "completed_today": false,
  "created_at": "2026-03-10T15:10:00Z"
}
```

If validation fails, the API returns `400 Bad Request` with an error body as described in **Error Responses**.

PUT `/api/habits/{id}/`

Update an existing habit. Only the fields provided in the request body are updated.

Path Parameters

- `id` (integer): Identifier of the habit to update.

Example Request Body

```
{
  "completed_today": true,
  "streak": 6
}
```

Example Successful Response

Status: `200 OK`

```
{
  "id": 1,
  "name": "Drink water",
  "frequency": "daily",
  "streak": 6,
  "completed_today": true,
  "created_at": "2026-03-05T08:00:00Z"
}
```

If the habit does not exist, the API returns `404 Not Found`. If the payload is invalid, it returns `400 Bad Request`.

DELETE `/api/habits/{id}/`

Delete a habit by its `id`.

Path Parameters

- `id` (integer): Identifier of the habit to delete.

Example Successful Response

Status: `204 No Content`

Response body: *(empty)*

If the habit does not exist, the API returns `404 Not Found`.

Filtering

Query parameters can be appended to list endpoints to filter the results returned by the API. This allows clients to retrieve only the subset of resources that match specific criteria.

Task Filtering

Tasks can be filtered by `status` and `priority`.

Supported Query Parameters

- `status` — filter by task status.
- Valid values: `not_started`, `in_progress`, `completed`
- `priority` — filter by task priority.
- Valid values: `low`, `medium`, `high`

These parameters can be used individually or combined.

Examples

- Filter tasks by status:

```
GET /api/tasks/?status=completed
```

- Filter tasks by priority:

```
GET /api/tasks/?priority=high
```

- Filter tasks by status and priority together:

```
GET /api/tasks/?status=in_progress&priority=medium
```

If an invalid value is provided (for example, `status=done`), the API responds with `400 Bad Request` and a JSON body describing the error.

Habit Filtering

Habits can be filtered by `frequency` and `completed_today`.

Supported Query Parameters

- `frequency` — filter by habit frequency.
- Valid values: `daily`, `weekly`
- `completed_today` — filter by whether the habit is completed today.
- Valid values: `true`, `false` (string values in the query string)

Examples

- Filter habits by frequency:

```
GET /api/habits/?frequency=daily
```

- Filter habits by completion status:

```
GET /api/habits/?completed_today=true
```

These parameters may be combined. If an invalid value is provided (for example, `completed_today=yes`), the API responds with `400 Bad Request` and a JSON body describing the error.

Error Responses

The API uses standard HTTP status codes to indicate the result of each request.

Status Codes

Status Code	Meaning
`200 OK`	The request was successful, and the response body contains the requested data.
`201 Created`	A new resource was successfully created.
`204 No Content`	The request was successful and there is no response body (typically for DELETE).
`400 Bad Request`	The request could not be processed due to validation errors or invalid input.

Status Code	Meaning
`404 Not Found`	The requested resource does not exist.
`405 Method Not Allowed`	The HTTP method is not supported for the requested endpoint.

Example Error Responses

Invalid JSON

If the request body is not valid JSON, the API returns `400 Bad Request`:

```
{
  "error": "Invalid JSON"
}
```

Validation Errors

If the JSON is valid but fails field validation (for example, a required field is missing or a value is invalid), the API returns `400 Bad Request` with an `errors` array:

```
{
  "errors": [
    "title is required"
  ]
}
```

Other validation messages may describe issues such as invalid dates, invalid priority or status values for tasks, or invalid frequency and streak values for habits.

Resource Not Found

If a client requests a task or habit that does not exist, the server returns:

- Status: `404 Not Found`
- Response body: *(Django default 404 response, typically HTML)*

In all cases, clients should check both the HTTP status code and the response body to understand the outcome of a request.